



Exercises session 3

PTIME

1 Definitions

Definition 1.1 Let $t : \mathbb{N} \rightarrow \mathbb{N}$ be a function. We define the (deterministic) **time complexity class** $DTIME(t(n))$ as:

$$DTIME(t(n)) = \{L \mid L \text{ is decided by an } O(t(n)) \text{ time deterministic Turing machine}\}.$$

Definition 1.2 $PTIME$ (or $\mathbf{P}$) is the class of all languages decidable in deterministic polynomial time on a single-tape TM

$$PTIME = \bigcup_k DTIME(n^k).$$

Definition 1.3 (Vertex colouring) A vertex colouring of a graph $G = (V, E)$ with k colours is a function

$$c : V(G) \rightarrow \{1, \dots, k\}$$

such that adjacent nodes have different colours, that is

$$(u, v) \in E \rightarrow c(u) \neq c(v)$$

2 Session exercises

1. Prove that $\mathbf{P}$ is closed under intersection, union, concatenation and complement.
2. Show that the following language

$$\text{TRIANGLE} = \{\langle G \rangle : G \text{ has a triangle}\}$$

is in $\mathbf{P}$

3. Consider the following problem:

$$2\text{-COL} = \{G : G \text{ has a 2-colouring}\}$$

- Design an algorithm that decides if a graph has a 2-colouring

- Relate this problem to 2-SAT and conclude that 2-COL is in PTIME.
4. We saw in classes that Horn-SAT is solvable in polynomial time. However, sometimes not all clauses can be satisfied. In such cases, we seek an assignment that satisfies the maximum number of clauses—this is the **Max-Horn-SAT** problem.
Given the following Horn formula:

$$\varphi = (\neg x_1 \vee x_2) \wedge (\neg x_2 \vee x_3) \wedge (\neg x_3 \vee x_4) \wedge (\neg x_4 \vee x_1) \wedge (\neg x_3) \wedge (x_4)$$

- (a) Determine if a satisfying assignment exists.
- (b) If the formula is unsatisfiable, determine the **maximum number of clauses** that can be satisfied.
- (c) Provide an example of an assignment that **maximizes satisfied clauses**.
- (d) Can you think of an efficient algorithm for solving Max-Horn-SAT?

Solutions

1. Prove that P is closed under intersection, union, concatenation and complement.

• Intersection

Being closed under intersections mean that given $L_1 \in P$ and $L_2 \in P$, then $L_1 \cap L_2 \in P$.

As $L_1 \in P$, then there exist a TM M_1 s.t. $L(M_1) = L_1$; also M_1 runs in polynomial time. As L_2 is also in P , there exist a TM M_2 s.t. $L(M_2) = L_2$ and M_2 also runs in polynomial time.

To decide if a word $w \in \Sigma^*$ is in $L_1 \cap L_2$, we need to check $w \in L_1$ and $w \in L_2$. As we can do this using M_1 and M_2 in polynomial time.

TO-DO: formalize a TM that decides $L_1 \cap L_2$

• Union

We need to prove that

$$L_1 \in P, L_2 \in P \implies L_1 \cup L_2 \in P.$$

As $L_1 \in P$, then there exist TM M_1 s.t. $L(M_1) = L_1$ and M_1 runs in polynomial time. Same with L_2 , there exists a TM M_2 s.t. $L(M_2) = L_2$ and M_2 runs in polynomial time.

In order to decide if $w \in \Sigma^*$ is in $L_1 \cup L_2$, then it should be that either $w \in L_1$, $w \in L_2$, or both. We can decide this in polynomial time using M_1 and M_2 , we can feed w to M_1 , if it accepts, we accept. If not, we feed it into M_2 , if it accepts, we accept, otherwise reject.

TO-DO: formalize this!

• Concatenation

A word $w \in \Sigma^*$ is in $L_1 \cdot L_2$ if $w = w_1 \cdot w_2$ where $w_1 \in L_1$ and $w_2 \in L_2$. We can do this as follows, let's assume $w = w_1 \dots w_m$. Let's also denote $w_0 = w_{m+1} = \epsilon$ (empty word)

For i in $[0, m]$:

check if $w_0 \dots w_i \in L_1$

check if $w_{i+1} \dots w_m \in L_2$

if both, accept.

reject.

TO-DO: prove that this runs in polynomial time!

• Complement.

w belongs to the complement of L if $w \notin L$.

We can decide this using the Turing machine for L , as follows.

if M accepts w :
 reject

else:
 accept.

as M runs in polynomial time, so does above procedure.

2. Show that the following language

$$\text{TRIANGLE} = \{ \langle G \rangle : G \text{ has a triangle} \}$$

is in P

given a graph $G = (V, E)$, (undirected)

a "triangle" is a set of vertices

$$v_1, v_2, v_3 \text{ s.t. } \begin{aligned} (v_1, v_2) \\ (v_2, v_3) \in E \\ (v_1, v_3) \end{aligned}$$

We can decide TRIANGLE as follows

Let $V = v_1, \dots, v_m$

- ① enumerate all tuples (v_i, v_j, v_k) ($i < j < k$) $\left\{ \Theta(|V|^3) \right.$
 FOR each tuple:
- ② check if $(v_i, v_j), (v_j, v_k), (v_i, v_k) \in E$ $\left\{ \Theta(|E|) \left(\Theta(|V|^2) \right) \right.$
- ③ if so, accept
- ④ reject.

Consider the following problem:

$$2\text{-COL} = \{G : G \text{ has a 2-colouring}\}$$

- Design an algorithm that decides if a graph has a 2-colouring
- Relate this problem to 2-SAT and conclude that 2-COL is in PTIME.

ALGORITHM

 claim: a graph has a 2-colouring if and only if it's bipartite.

FOR YOU: prove the claim.

The algorithm goes as follow:

while not all nodes are visited:

① pick an unvisited node u , assign color 1

② for every v s.t. $(u,v) \in E$
• assign v with color 2.
→ if v was visited with color 1, reject.

is the algorithm correct?

YES! → it checks for bipartiteness.

② 2-COL IS IN PTIME

the idea here is to "transform" an instance of 2-COL into an instance of 2-SAT.

given $G = (V, E)$

- for $v \in V$, create variable x_v
- for $(u, v) \in E$, create $(x_u \vee x_v)$
and $(\neg x_u \vee \neg x_v)$

claims:

- ① we can construct the formula in polynomial time
- ② φ_G is satisfiable $\iff G$ is 2 colorable
 - color $\sim$ truth assignment

4. We saw in classes that Horn-SAT is solvable in polynomial time. However, sometimes not all clauses can be satisfied. In such cases, we seek an assignment that satisfies the maximum number of clauses—this is the **Max-Horn-SAT** problem.

Given the following Horn formula:

$$\varphi = (\neg x_1 \vee x_2) \wedge (\neg x_2 \vee x_3) \wedge (\neg x_3 \vee x_4) \wedge (\neg x_4 \vee x_1) \wedge (\neg x_3) \wedge (x_4)$$

- (a) Determine if a satisfying assignment exists.
- (b) If the formula is unsatisfiable, determine the **maximum number of clauses** that can be satisfied.
- (c) Provide an example of an assignment that **maximizes satisfied clauses**.
- (d) Can you think of an efficient algorithm for solving Max-Horn-SAT?

(a) determine if φ is satisfiable

0. $(\neg x_1 \vee x_2) \wedge (\neg x_2 \vee x_3) \wedge (\neg x_3 \vee x_4)$
 $\wedge (\neg x_4 \vee x_1) \wedge (\neg x_3) \wedge (x_4)$

$$x_4 = \text{TRUE}$$

$$x_3 = \text{FALSE}$$

1. $(\neg x_1 \vee x_2) \wedge (\neg x_2) \wedge (x_1)$

$$x_2 = \text{FALSE}$$

$$x_1 = \text{TRUE}$$

$$0 \quad \times$$

the formula is not satisfiable

b. Finding a "max" assignment

In the case before, just one clause was not satisfied, so assignment was automatically "maximal"

c. max assignment is

$$x_1 = \text{TRUE}$$

$$x_2 = \text{FALSE}$$

$$x_3 = \text{FALSE}$$

$$x_4 = \text{TRUE}$$

d. a general algorithm

Can we leverage the structure of 3-SAT clauses to create a general algorithm that runs in PTIME?

↳ if you succeed;
congratulations!
you have proven that
 $P = NP$.